

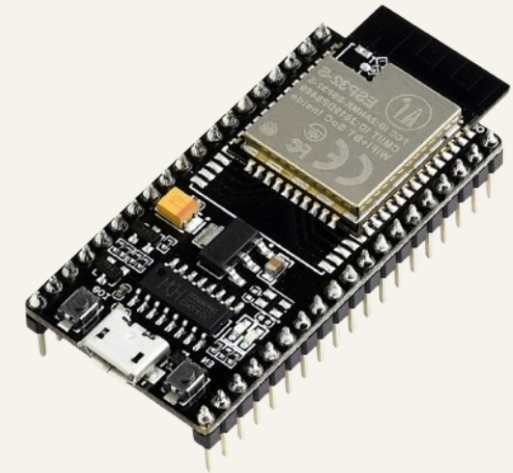
Remotely controlled tank

Wireless manual control with explicit software safety states.
Fully functional, network-controlled robotic vehicle with articulated movement and precise aiming capabilities.
Featuring a custom 3D printed case for the electronic components.

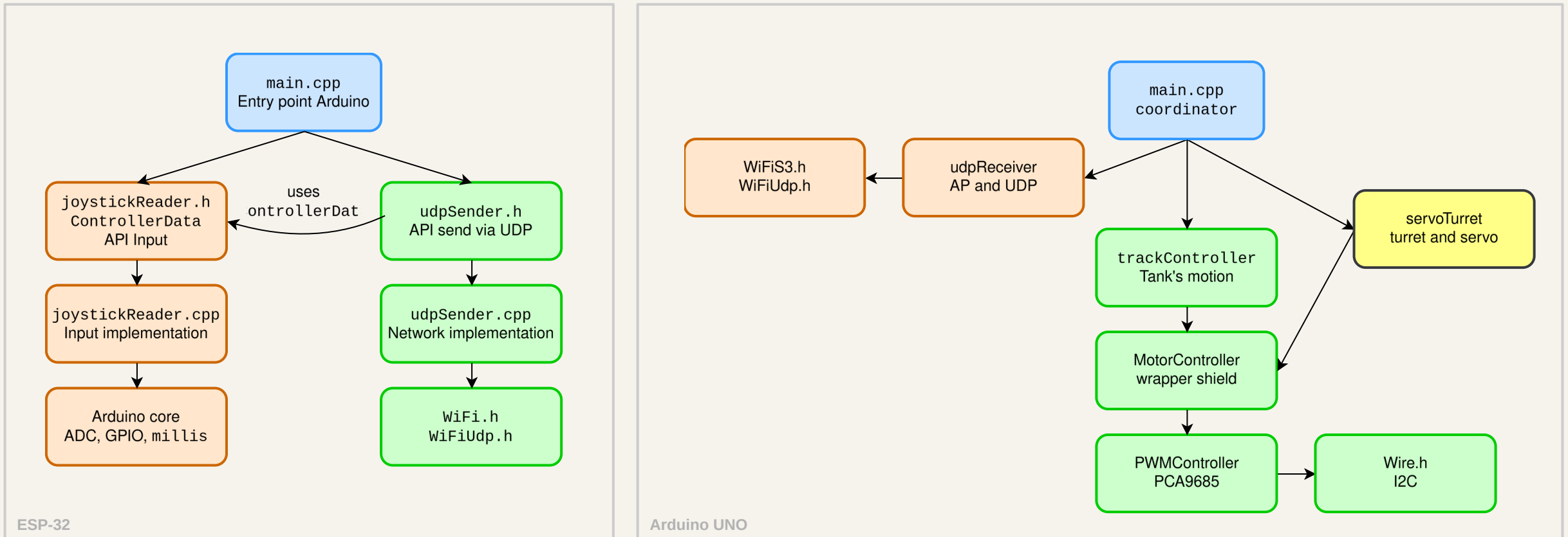


Hardware components overview

- **Arduino Uno R4 WiFi:** Main microcontroller and web server host.
- **ESP32-DevKitC-32UE:** Microcontroller used for the remote control.
- **EmakeFun PS2X & Motor Drive Board:** Primary motor driver interface.
- **DC Motors (2x):** Independent track movement.
- **Servomotors (2x):** Vertical inclination of the cannon muzzle (maximum vertical rotation: 45°).
- **Stepper Motor & Driver:** Lateral/horizontal rotation of the turret, allowing a rotation of 45° on both sides.
- **HV Electromagnetic Cannon DIY Kit:** Main firing mechanism (DC3V).
- **Battery:** 3300mAh capacity, 7.4V.



Architecture overview



Code showcase

```

1 void UdpSender_send(const ControllerData& data) {
2     if (!ensureWifiConnected())
3         return;
4
5     if (suppressCommandAfterConnection) {
6         suppressCommandAfterConnection = false;
7         return;
8     }
9
10    if (udpTxBackoffActive(millis()))
11        return;
12
13    char message[64];
14    int messageLength =
15        snprintf(message, sizeof(message), "V1;%d;%d;%d;%d;%d;%d", data.driveX, data.driveY,
16            data.turretX, data.elevationY, data.zeroPressed ? 1 : 0, data.firePressed ? 1 : 0);
17
18    if (messageLength <= 0 || messageLength >= static_cast<int>(sizeof(message))) {
19        Serial.println("ERROR: pacchetto UDP controller non valido");
20        return;
21    }
22
23    if (udp.beginPacket(tankIP, UDP_PORT) == 0) {
24        registerUdpSendFailure();
25        return;
26    }
27
28    size_t writtenBytes = udp.print(message);
29    int packetSent = udp.endPacket();
30    if (writtenBytes != static_cast<size_t>(messageLength) || packetSent != 1) {
31        registerUdpSendFailure();
32        return;
33    }
34 }
35 }

```

controller-esp32/src/udpSender.cpp

```

1 void setup() {
2     int inactiveRelayLevel = FIRE_RELAY_ACTIVE_HIGH ? LOW : HIGH;
3     digitalWrite(FIRE_RELAY_PIN, inactiveRelayLevel);
4     pinMode(FIRE_RELAY_PIN, OUTPUT);
5     setFireRelay(false);
6     digitalWrite(TURRET_DRIVER_A_PIN, LOW);
7     digitalWrite(TURRET_DRIVER_B_PIN, LOW);
8     digitalWrite(TURRET_DRIVER_C_PIN, LOW);
9     digitalWrite(TURRET_DRIVER_D_PIN, LOW);
10    StepperTorretta_begin();
11    Serial.begin(9600);
12    Wire.begin();
13
14    shieldReady = configureShieldAddress();
15    if (shieldReady) {
16        shieldReady = shield.begin();
17        if (shieldReady) {
18            shield.DCbrakeAll();
19            shieldReady = shield.isCommunicationHealthy();
20        }
21
22        if (shieldReady) {
23            TrackController_begin(shield);
24            ServoTorretta_begin(shield);
25            shieldReady = shield.isCommunicationHealthy();
26        }
27
28        if (!shieldReady) {
29            shieldFaultReported = true;
30            Serial.println("Error: PCA9685 init failed!");
31        }
32    }
33
34    delay(1000);
35    UdpReceiver_begin();
36 }

```

tank/src/main.cpp

Testing

- **Physical device:** checking if actual result = desired result;
- **Serial monitor:** checking log messages using “pio device monitor” command;
- **Wireshark:** UDP packet sniffing to check protocol integrity.

Results

- **Physical device:** tank works as intended;
- **UDP protocol:** works fine in low-traffic networks; delays happen only if the device is around networks with 20+ devices.

Improvements and future work

01

WEB INTERFACE

Basic HTML/CSS dashboard using Arduino Uno R4 WiFi's web server capabilities. Adds flexibility and is easier to debug.

02

FPV CAMERA

Adding an ESP32-CAM or a dedicated FPV camera for better aiming and driving. The video feed would be separate from the control data (Wi-Fi for control, analog/2.4GHz for video).

03

TARGET DISTANCE MEASUREMENT

Using an ultrasound sensor and an accelerometer to calculate the tank's distance from the target and turret incline.

04

AUTOMATED FIRING SYSTEM

Implementation of a state machine for automatic loading and firing of the cannon.

05

UDP V2

Congestion control, Forward Error Correction (FEC), and multipath support.

Thanks for the attention

Team members:

- Christian BELLA
- Selmir KUSI
- Francesco MARTELLA
- Alice SEDIOLI

